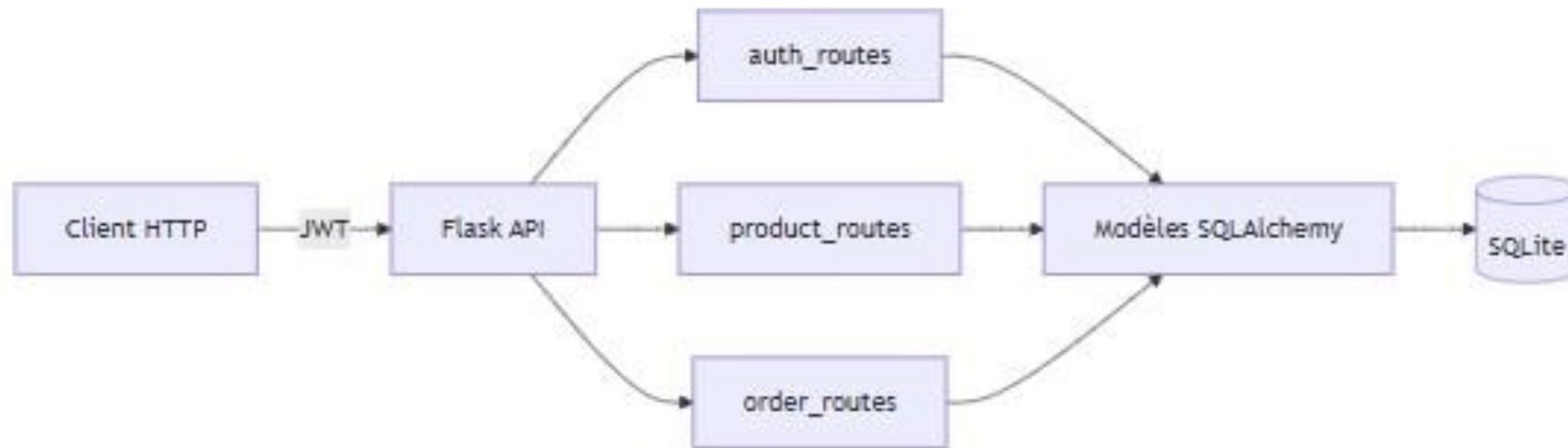
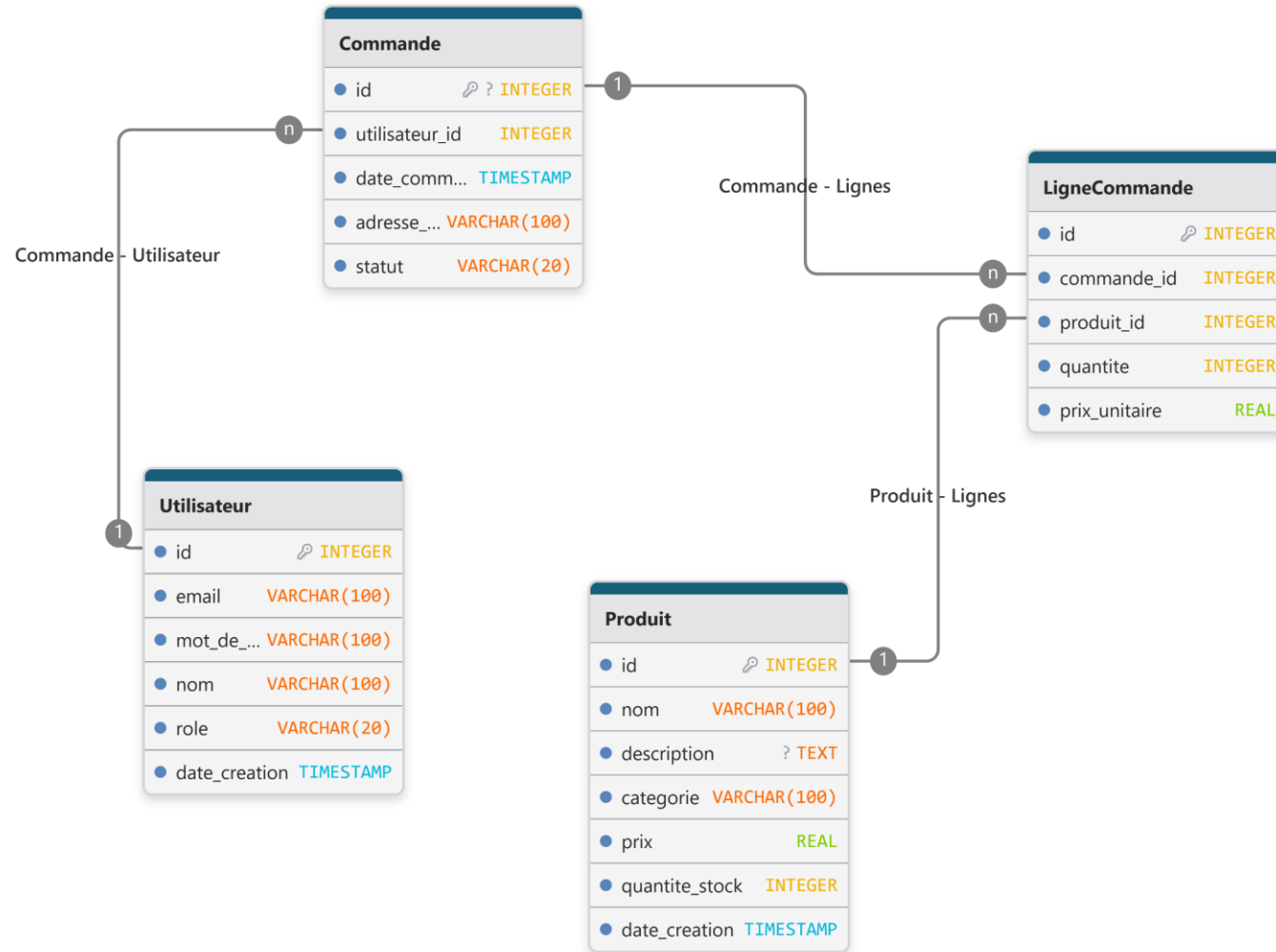


# Structure du repo / projet

```
.
├── src/
│   ├── app.py          # Point d'entrée Flask, enregistrement des blueprints
│   ├── models.py       # Modèles SQLAlchemy (Utilisateur, Produit, Commande, LigneCommande)
│   ├── utils.py        # Décorateurs JWT, sérialiseurs, validations
│   ├── db_init.py      # Script de seed (reset + données de test)
│   └── routes/         # Blueprints : auth_routes, product_routes, order_routes
│       └── swagger/    # Fichiers de documentation Swagger par route
├── tests/              # Tests pytest (auth, produits, commandes, utils)
├── docs/               # Documentation (schéma SQL, captures)
├── .github/workflows/  # CI (lint + tests) et CD (build Docker + déploiement)
├── Dockerfile
└── pyproject.toml
```



# Structure de la base de données



- Personnalisation de l'exemple Digishop pour en faire un cas d'usage plus orienté vers une base de e-commerce santé = « ParaShop ».

# Routes API Flask & Endpoints

Swagger

/apispec\_1.json

Explore

ParaShop API

1.0.0

/apispec\_1.json

API REST de la plateforme e-commerce ParaShop.

Authorize

Authentification

POST /api/auth/login Authentifier un utilisateur et obtenir un token JWT. post\_api\_auth\_login

POST /api/auth/register Créer un nouveau compte utilisateur. post\_api\_auth\_register

Commandes

POST /api/commandes Créer une nouvelle commande pour l'utilisateur connecté. post\_api\_commandes

GET /api/commandes/ Lister les commandes. Un admin voit toutes les commandes, un client voit les siennes. get\_api\_commandes\_

GET /api/commandes/{id} Récupérer le détail d'une commande par son identifiant. get\_api\_commandes\_id\_

PATCH /api/commandes/{id} Modifier le statut d'une commande (admin uniquement). patch\_api\_commandes\_id\_

GET /api/commandes/{id}/lignes Récupérer les lignes de commande d'une commande. get\_api\_commandes\_id\_lignes

Produits

GET /api/produits Lister tous les produits du catalogue. get\_api\_produits

POST /api/produits Créer un nouveau produit (admin uniquement). post\_api\_produits

GET /api/produits/search Rechercher des produits par nom et/ou catégorie. get\_api\_produits\_search

DELETE /api/produits/{id} Supprimer un produit (admin uniquement). delete\_api\_produits\_id\_

Ajout d'un swagger pour plus de facilité concernant la documentation

## Endpoints

### Authentification — /api/auth

| Méthode | Route              | Description                   | Auth |
|---------|--------------------|-------------------------------|------|
| POST    | /api/auth/register | Créer un compte               | Non  |
| POST    | /api/auth/login    | Se connecter, retourne un JWT | Non  |

### Produits — /api/produits

| Méthode | Route                                | Description                   | Auth  |
|---------|--------------------------------------|-------------------------------|-------|
| GET     | /api/produits                        | Liste tous les produits       | JWT   |
| GET     | /api/produits/<id>                   | Détail d'un produit           | JWT   |
| GET     | /api/produits/search?nom=&categorie= | Recherche par nom / catégorie | JWT   |
| POST    | /api/produits                        | Créer un produit              | Admin |
| PUT     | /api/produits/<id>                   | Modifier un produit           | Admin |
| DELETE  | /api/produits/<id>                   | Supprimer un produit          | Admin |

### Commandes — /api/commandes

| Méthode | Route                      | Description                                              | Auth  |
|---------|----------------------------|----------------------------------------------------------|-------|
| GET     | /api/commandes             | Liste les commandes (toutes si admin, les siennes sinon) | JWT   |
| GET     | /api/commandes/<id>        | Détail d'une commande                                    | JWT   |
| POST    | /api/commandes             | Passer une commande                                      | JWT   |
| PATCH   | /api/commandes/<id>        | Modifier le statut d'une commande                        | Admin |
| GET     | /api/commandes/<id>/lignes | Lignes d'une commande                                    | JWT   |

# Gestion des exceptions – Règles métiers

| Catégorie         | Route                     | Rôle  | Règles - Exceptions                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------|---------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentification  | POST /api/auth/register   | All   | <ul style="list-style-type: none"> <li>✓ Utilisateur déjà présent dans la base</li> <li>✓ Information(s) requise(s) non renseignée(s) (email, nom, mot_de_passe)</li> <li>✓ Structure email conforme (regex)</li> <li>✓ Présence d'un caractère spécial, un chiffre, une majuscule dans le mdp</li> <li>✓ Création du rôle admin uniquement si le bon « secret » se trouve dans la requête</li> </ul> |
|                   | POST /api/auth/login      | All   | <ul style="list-style-type: none"> <li>✓ Email inexistant dans la base</li> <li>✓ Présence des informations (email, mot_de_passe)</li> <li>✓ Vérification conformité du mdp</li> </ul>                                                                                                                                                                                                                |
| Catalogue produit | GET /api/produits         | All   | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification</li> </ul>                                                                                                                                                                                                                                                                                                              |
|                   | GET /api/produits/{id}    | All   | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification</li> <li>✓ Existence du produit dans la base</li> </ul>                                                                                                                                                                                                                                                                 |
|                   | GET /api/produits/search  | All   | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification</li> </ul>                                                                                                                                                                                                                                                                                                              |
|                   | POST /api/produits        | Admin | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification + rôle admin</li> <li>✓ Vérifie que le produit n'est pas déjà existant dans la base</li> <li>✓ Vérifie que informations requises sont présentes</li> <li>✓ Vérifie que informations requises sont au bon format</li> </ul>                                                                                              |
|                   | PUT /api/produits/{id}    | Admin | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification + rôle admin</li> <li>✓ Vérifie que le produit existe dans la base</li> <li>✓ Vérifie que informations requises sont présentes</li> <li>✓ Vérifie que informations requises sont au bon format</li> </ul>                                                                                                               |
|                   | DELETE /api/produits/{id} | Admin | <ul style="list-style-type: none"> <li>✓ Conformité du token d'identification + rôle admin</li> <li>✓ Vérifie que le produit existe dans la base</li> <li>✓ Vérifie si le produit n'est pas en cours dans une commande</li> </ul>                                                                                                                                                                     |

# Gestion des exceptions – Règles métiers (2)

| Catégorie             | Route                          | Rôle  | Règles - Exceptions                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|--------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gestion des commandes | GET /api/commandes             | All   | <input checked="" type="checkbox"/> Conformité du token d'identification                                                                                                                                                                                                                                                                                            |
|                       | GET /api/commandes/{id}        | All   | <input checked="" type="checkbox"/> Conformité du token d'identification<br><input checked="" type="checkbox"/> Existence de la commande dans la base<br><input checked="" type="checkbox"/> Interdiction d'accès aux commandes des autres clients                                                                                                                  |
|                       | POST /api/commandes            | Admin | <input checked="" type="checkbox"/> Conformité du token d'identification<br><input checked="" type="checkbox"/> Vérifie que informations requises sont présentes<br><input checked="" type="checkbox"/> Vérifie que informations requises sont au bon format<br><input checked="" type="checkbox"/> Vérification d'un niveau de stock suffisant → MAJ quantités     |
|                       | PATCH /api/commandes/{id}      | Admin | <input checked="" type="checkbox"/> Conformité du token d'identification<br><input checked="" type="checkbox"/> Vérification du statut initial pour s'assurer qu'il change bien<br><input checked="" type="checkbox"/> Vérifie que informations requises sont présentes<br><input checked="" type="checkbox"/> Vérifie que informations requises sont au bon format |
|                       | GET /api/commandes/{id}/lignes | Admin | <input checked="" type="checkbox"/> Conformité du token d'identification                                                                                                                                                                                                                                                                                            |

# Couverture de tests unitaires

Coverage report: 60%

Files

Functions

Classes

coverage.py v7.14.0, created at 2026-05-16 11:52 +0200

| File ▲                           | statements | missing    | excluded | coverage   |
|----------------------------------|------------|------------|----------|------------|
| src \ __init__.py                | 0          | 0          | 0        | 100%       |
| src \ app.py                     | 22         | 1          | 0        | 95%        |
| src \ db_init.py                 | 43         | 43         | 0        | 0%         |
| src \ models.py                  | 52         | 5          | 0        | 90%        |
| src \ routes \ __init__.py       | 0          | 0          | 0        | 100%       |
| src \ routes \ auth_routes.py    | 49         | 10         | 0        | 80%        |
| src \ routes \ order_routes.py   | 116        | 56         | 0        | 52%        |
| src \ routes \ product_routes.py | 99         | 49         | 0        | 51%        |
| src \ utils.py                   | 59         | 10         | 0        | 83%        |
| <b>Total</b>                     | <b>440</b> | <b>174</b> | <b>0</b> | <b>60%</b> |

coverage.py v7.14.0, created at 2026-05-16 11:52 +0200

- Une vingtaine de tests unitaires (positifs ou négatifs) ont été mis en œuvre pour garantir le bon fonctionnement des fonctionnalités principales.

| Couche                                          | Nb de tests | Couverture                                              |
|-------------------------------------------------|-------------|---------------------------------------------------------|
| Auth / JWT (test_auth.py)                       | 3           | Inscription + connexion (heureux + mauvais mdp)         |
| Produits HTTP (test_products.py)                | 6           | GET liste/détail, POST création, PUT sécurité, négatifs |
| Produit modèle/sérialisation (test_products.py) | 2           | Attributs + produit_to_dict                             |
| Commandes (test_orders.py)                      | 3           | Création + règle métier de stock                        |
| Utils purs (test_utils.py)                      | 6           | check_fields, is_valid_email, is_valid_password         |

- Lancement à la demande : `uv run pytest`  
Ainsi qu'à chaque push (CI/CD)

# Orientations CI/CD

```
Dockerfile > ...
1  # Image de base officielle Python 3.11 en version slim
2  FROM python:3.11-slim
3
4  #répertoire de travail à l'intérieur du conteneur
5  WORKDIR /app
6
7  # Copie des fichiers de dépendances en premier (cache Docker)
8  COPY pyproject.toml uv.lock README.md ./
9  COPY src/ ./src/
10 COPY entrypoint.sh ./
11
12 # Installe uv puis synchronise les dépendances depuis uv.lock
13 # --no-dev : exclut pytest, ruff et les autres outils de développement
14 # --system : installe dans le Python système (pas de venv dans le conteneur)
15 RUN pip install uv && \
16     uv sync --no-dev --system && \
17     chmod +x entrypoint.sh && \
18     mkdir -p data
19
20 # Port imposé par la consigne
21 EXPOSE 5000
22
23 # entrypoint.sh initialise la base au premier démarrage, puis lance gunicorn
24 ENTRYPOINT ["./entrypoint.sh"]
25
```

Conteneurisation - Dockerfile

1. Linting & formatage unifiés avec Ruff, Un seul outil (Ruff v0.6.9) remplace flake8 + black + isort, configuré dans pyproject.toml (IDE, pre-commit hook et CI)
2. Suite de tests unitaires structurée avec pytest, 20 tests
3. Pre-commit comme garde-fou de sécurité 8 hooks bloquent les erreurs avant le commit :
  - Ruff (lint + format)
  - Gitleaks
  - Check-added-large-files
  - Check-yaml
4. Dépendances reproductibles avec uv + pyproject.toml, ce qui rend l'image légère et la config déterministe
5. Pipeline CI GitHub Actions — Workflow `.github/workflows/ci.yml` déclenché sur push vers main